

COMPSCI 689

Lecture 24: Automatic Differentiation

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

Introduction

```
#Init model parameters
w = torch.randn(5,1, requires_grad=True)

# Compute output and loss
yhat = x@w
loss = torch.mean((y-yhat)**2)

# Compute gradient
loss.backward()
w.grad
```

[221] ✓ 0.0s

```
... tensor([[ -2.3204],
          [-1.7099],
          [ 2.1330],
          [ 1.0153],
          [ 4.3975]])
```

Approaches to Computing Gradients

Function (Math)

$$L(D, w) = \frac{1}{N} \sum_{n=1}^N (y_n - x_n w)^2$$

Manual
Coding

Function (Code):

```
def loss(x, y, w):  
    yhat = x@w  
    return torch.mean((y-yhat)**2)
```

Manual
Differentiation

Gradient (Math)

$$\nabla L(D, w) = -2 \frac{1}{N} \sum_{n=1}^N (y_n - x_n w) x_n^T$$

Manual
Coding

Gradient (Code):

```
def grad(x, y, w):  
    err = x@w-y  
    g = 2*err*x  
    return g.mean(axis=0, keepdims=True).T
```

Function (Math)

$$L(D, w) = \frac{1}{N} \sum_{n=1}^N (y_n - x_n w)^2$$

Manual
Coding



Function (Code):

```
def loss(x,y,w):  
    yhat = x@w  
    return torch.mean((y-yhat)**2)
```

Gradient (Code):

```
def grad(x,y,w):  
    err = x@w-y  
    g = 2*err*x  
    return g.mean(axis=0,keepdims=True).T
```



Symbolic
Differentiation

Table 1: Iterations of the logistic map $l_{n+1} = 4l_n(1 - l_n)$, $l_1 = x$ and the corresponding derivatives of l_n with respect to x , illustrating expression swell.

n	l_n	$\frac{d}{dx}l_n$	$\frac{d}{dx}l_n$ (Simplified form)
1	x	1	1
2	$4x(1 - x)$	$4(1 - x) - 4x$	$4 - 8x$
3	$16x(1 - x)(1 - 2x)^2$	$16(1 - x)(1 - 2x)^2 - 16x(1 - 2x)^2 - 64x(1 - x)(1 - 2x)$	$16(1 - 10x + 24x^2 - 16x^3)$
4	$64x(1 - x)(1 - 2x)^2(1 - 8x + 8x^2)^2$	$128x(1 - x)(-8 + 16x)(1 - 2x)^2(1 - 8x + 8x^2) + 64(1 - x)(1 - 2x)^2(1 - 8x + 8x^2)^2 - 64x(1 - 2x)^2(1 - 8x + 8x^2)^2 - 256x(1 - x)(1 - 2x)(1 - 8x + 8x^2)^2$	$64(1 - 42x + 504x^2 - 2640x^3 + 7040x^4 - 9984x^5 + 7168x^6 - 2048x^7)$

Function (Math)

$$L(D, w) = \frac{1}{N} \sum_{n=1}^N (y_n - x_n w)^2$$

Manual
Coding



Function (Code):

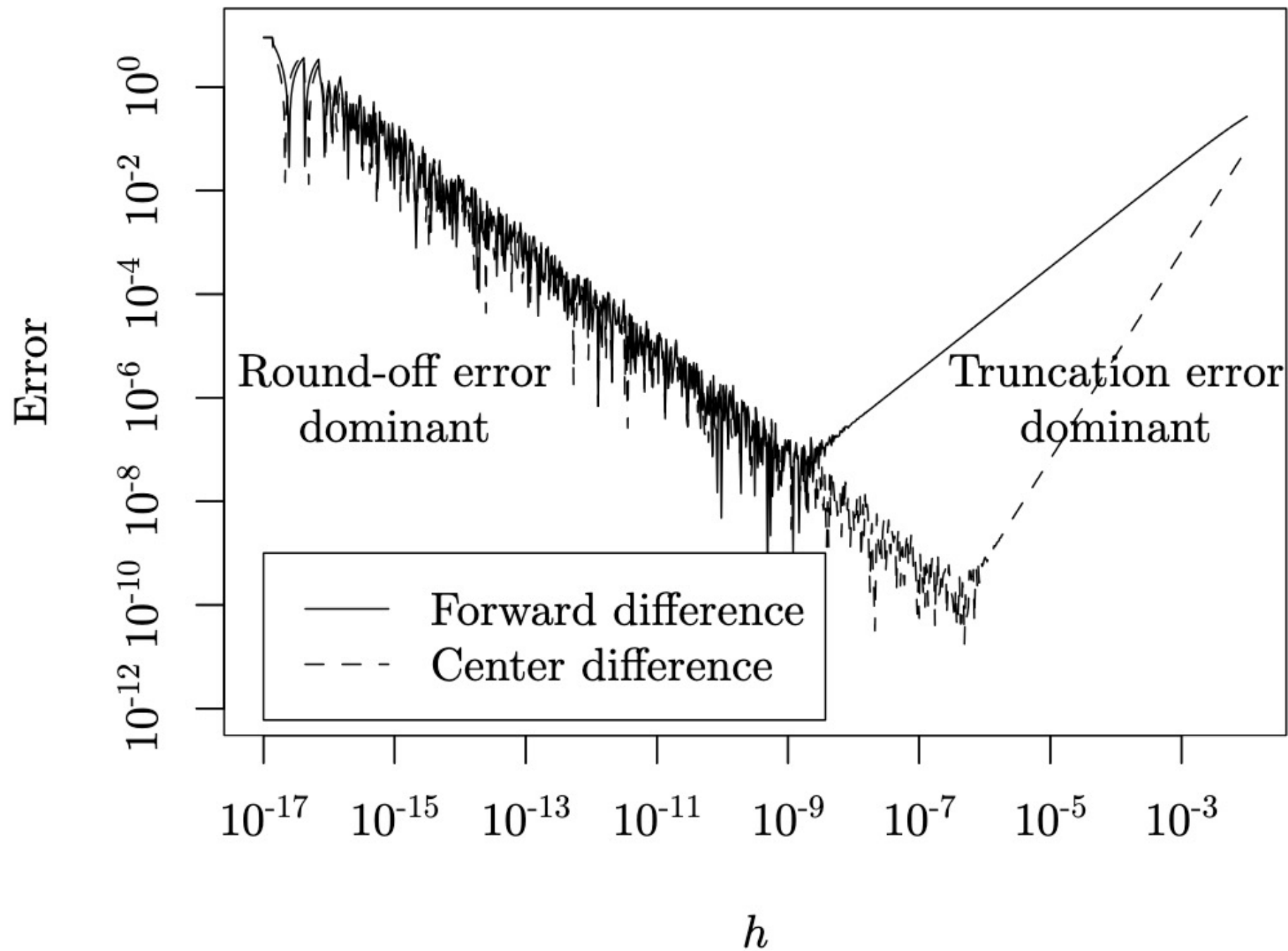
```
def loss(x,y,w):  
    yhat = x@w  
    return torch.mean((y-yhat)**2)
```

Gradient (Numerical):

```
def num_grad(x,y,w,eps=1e-5):  
    l=loss(x,y,w)  
    grad = torch.zeros(w.shape[0],1)  
    for i in range(w.shape[0]):  
        w2 = w.clone(); w2[i]+=eps  
        grad[i] = (loss(x,y,w2)-l)/eps  
    return grad
```



Numerical
Differentiation



Function (Math)

$$L(D, w) = \frac{1}{N} \sum_{n=1}^N (y_n - x_n w)^2$$

Manual
Coding



Function (Code):

```
def loss(x,y,w):  
    yhat = x@w  
    return torch.mean((y-yhat)**2)
```

Gradient (AutoDiff):

```
MeanBackward0:0  
└─ PowBackward0:0  
   └─ SubBackward0:0  
      └─ MmBackward0:1
```



Automatic
Differentiation

Forward Mode Autodiff

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$$\begin{aligned} \frac{\partial f(x_1, x_2)}{\partial x_1} &= \frac{1}{x_1 \cdot x_2}(x_2) \cdot \sin(x_2) \\ &= \frac{\sin(x_2)}{x_1} \end{aligned}$$

$$\begin{aligned} \frac{\partial f(x_1, x_2)}{\partial x_2} &= \frac{1}{x_1 \cdot x_2}(x_1) \cdot \sin(x_2) + \log(x_1 \cdot x_2) \cdot \cos(x_2) \\ &= \frac{\sin(x_2)}{x_2} + \log(x_1 \cdot x_2) \cdot \cos(x_2) \end{aligned}$$

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$a = x_1 \cdot x_2$	⋮	$\frac{\partial w}{\partial x_1} = \frac{\partial w}{\partial a} \frac{\partial a}{\partial x_1}$
$y_1 = \log(a)$		
$y_2 = \sin(x_2)$		$= \frac{\partial w}{\partial a}(x_2)$
$w = y_1 \cdot y_2$		

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$a = x_1 \cdot x_2$	⋮	$\frac{\partial w}{\partial a} = \frac{\partial w}{\partial y_1} \frac{\partial y_1}{\partial a}$
$y_1 = \log(a)$		$= \frac{\partial w}{\partial y_1} \frac{1}{a}$
$y_2 = \sin(x_2)$		
$w = y_1 \cdot y_2$		

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$$a = x_1 \cdot x_2$$

$$y_1 = \log(a)$$

$$y_2 = \sin(x_2)$$

$$w = y_1 \cdot y_2$$

.....

$$\frac{\partial w}{\partial y_1} = y_2$$

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$a = x_1 \cdot x_2$	⋮	$\frac{\partial w}{\partial x_1} = (y_2) \left(\frac{1}{a} \right) (x_2)$
$y_1 = \log(a)$		
$y_2 = \sin(x_2)$		
$w = y_1 \cdot y_2$		

Reverse Mode Autodiff

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$a = x_1 \cdot x_2$	⋮	$\frac{\partial w}{\partial x_1} = \frac{\partial w}{\partial y_1} \frac{\partial y_1}{\partial x_1} + \frac{\partial w}{\partial y_2} \frac{\partial y_2}{\partial x_1}$
$y_1 = \log(a)$		
$y_2 = \sin(x_2)$		$= (y_2) \frac{\partial y_1}{\partial x_1} + (y_1) \frac{\partial y_2}{\partial x_1}$
$w = y_1 \cdot y_2$		

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$a = x_1 \cdot x_2$	⋮	$\frac{\partial y_1}{\partial x_1} = \frac{\partial y_1}{\partial a} \frac{\partial a}{\partial x_1}$
$y_1 = \log(a)$		$= \left(\frac{1}{a}\right) \frac{\partial a}{\partial x_1}$
$y_2 = \sin(x_2)$		
$w = y_1 \cdot y_2$		$\frac{\partial a}{\partial x_1} = x_2$

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$$\begin{array}{l|l} a = x_1 \cdot x_2 & \frac{\partial y_2}{\partial x_1} = 0 \\ y_1 = \log(a) & \\ y_2 = \sin(x_2) & \\ w = y_1 \cdot y_2 & \end{array}$$

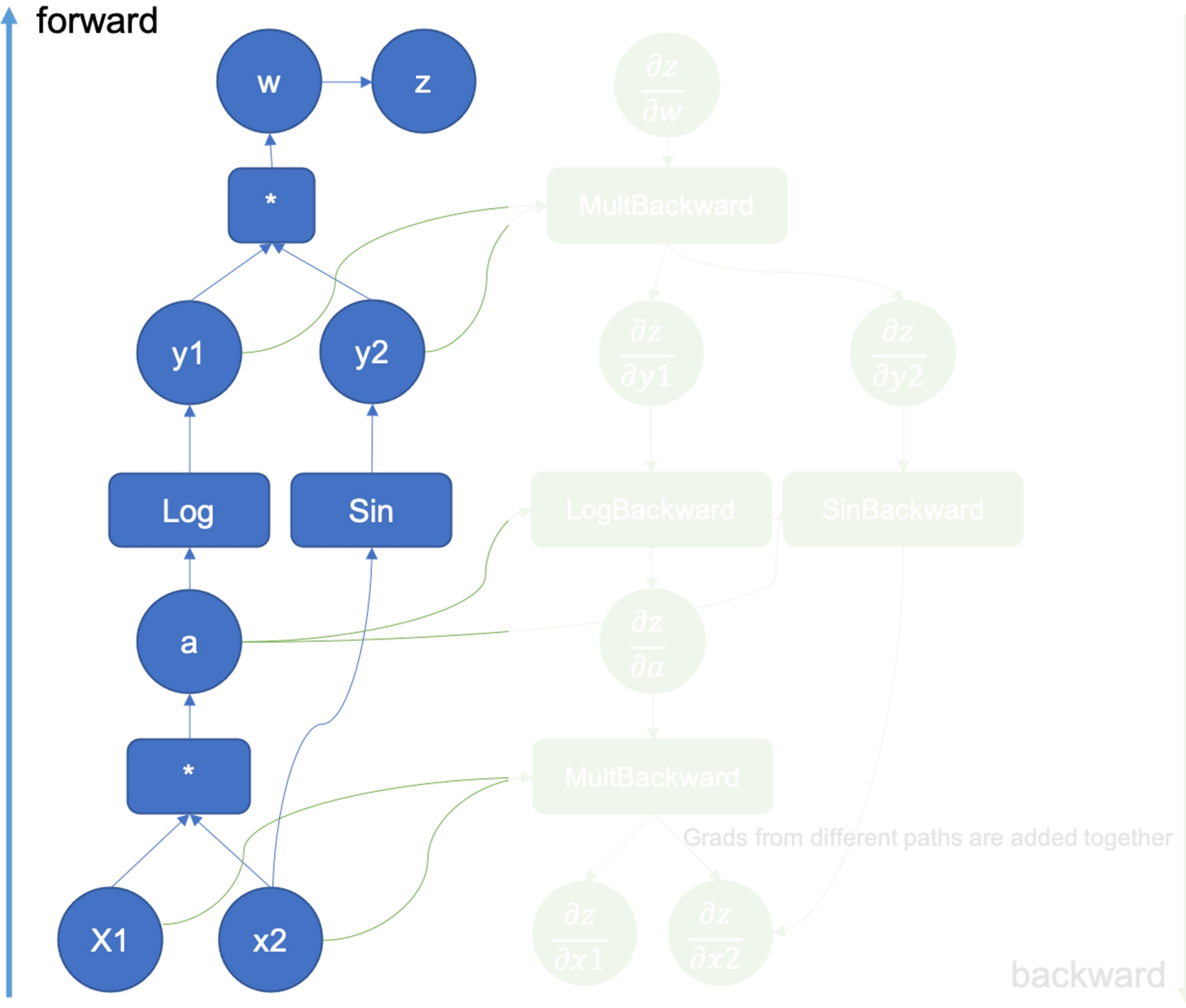
$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

$$\begin{array}{l} a = x_1 \cdot x_2 \\ y_1 = \log(a) \\ y_2 = \sin(x_2) \\ w = y_1 \cdot y_2 \end{array} \left| \frac{\partial w}{\partial x_1} = (y_2) \left(\frac{1}{a} \right) (x_2) + (y_1)(0) \right.$$

Reverse Mode Autodiff in PyTorch

$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

```
def fun(x1,x2):
    a = x1*x2
    y1 = torch.log(a)
    y2 = torch.sin(x2)
    return y1*y2
```





```
trace = torch.jit.trace(fun, (x1,x2))
g=str(trace.graph)
for n in g.split("\n"):
    | print(n.split("#")[0])
```

[179]

✓ 0.0s

```
... graph(%x1 : Float(requires_grad=1, device=cpu),
        %x2 : Float(requires_grad=1, device=cpu)):
    %a : Float(requires_grad=1, device=cpu) = aten::mul(%x2, %x1)
    %y1 : Float(requires_grad=1, device=cpu) = aten::log(%a)
    %y2 : Float(requires_grad=1, device=cpu) = aten::sin(%x2)
    %5 : Float(requires_grad=1, device=cpu) = aten::mul(%y1, %y2)
    return (%5)
```

Search nodes (regex)



Fit to screen



Download PNG



Upload file

Run (1)

Tag (2) Default

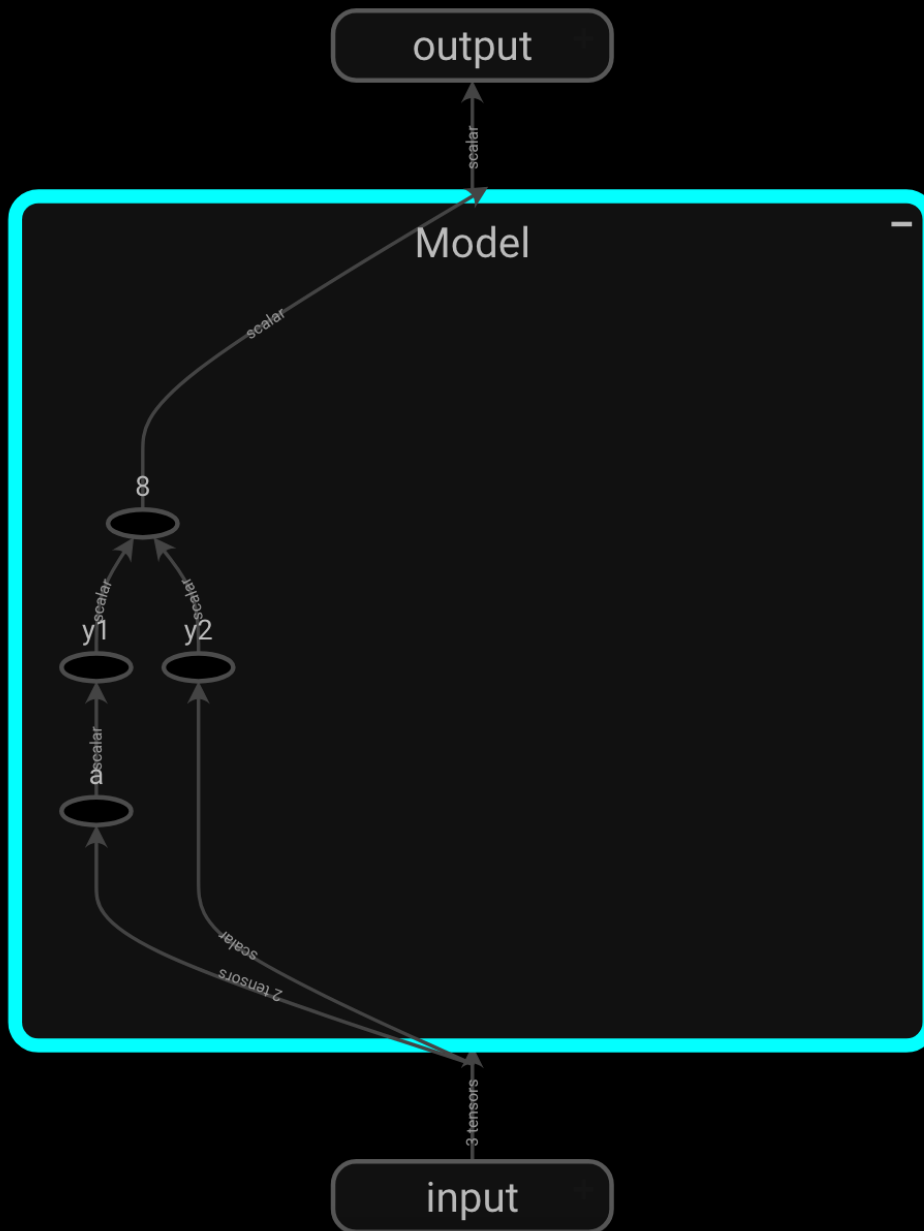
Graph type

Legend

- colors
- same substructure
 - unique substructure
 - unique substructure (* = expandable)
 - Namespace* ?
 - OpNode ?
 - Unconnected series* ?
 - Connected series* ?
 - Constant ?
 - Summary ?
 - Dataflow edge ?
 - Control dependency edge ?
 - Reference edge ?

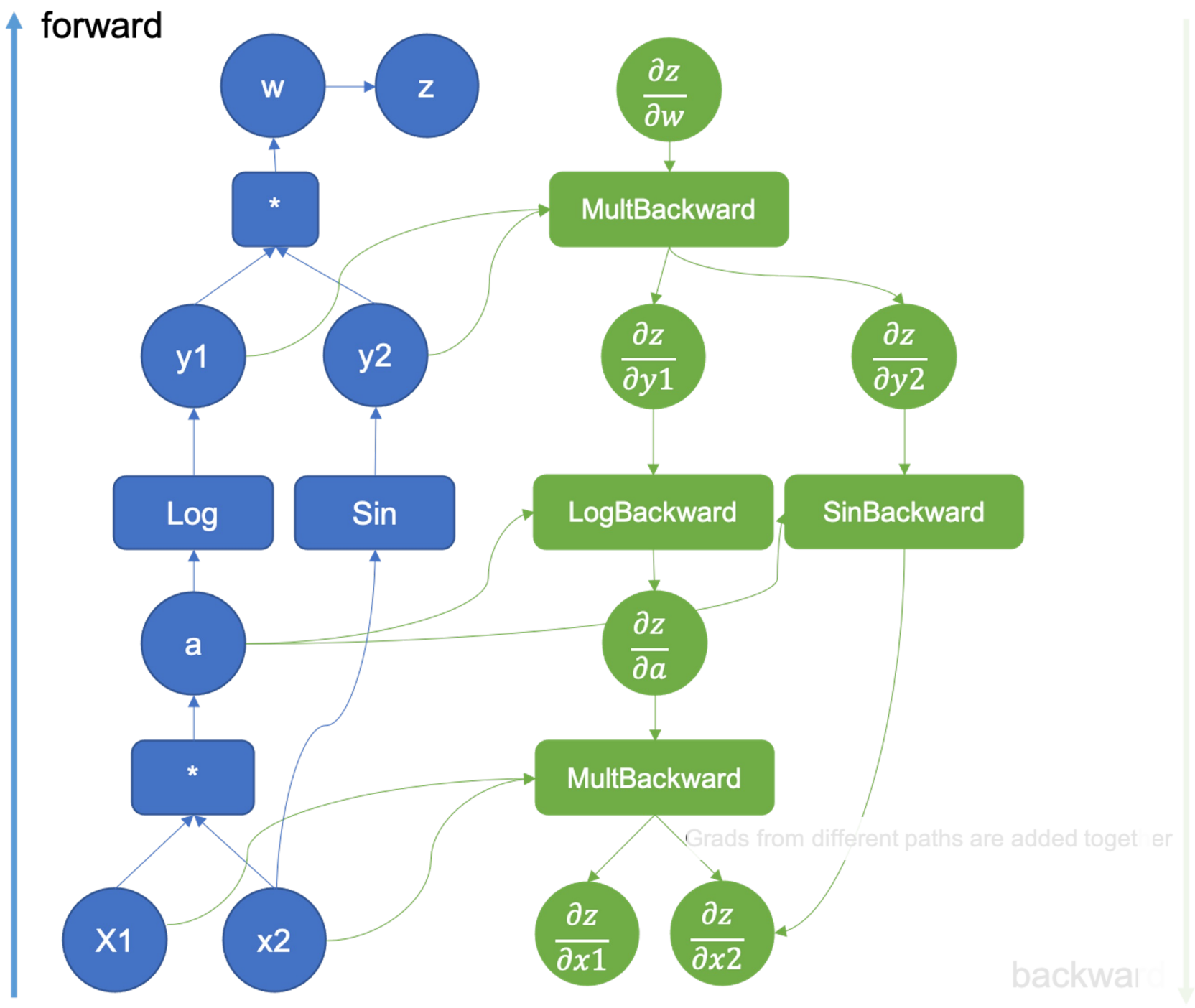
Model

Subgraph: 4 nodes



$$f(x_1, x_2) = \log(x_1 \cdot x_2) \cdot \sin(x_2)$$

```
def fun(x1,x2):
    a = x1*x2
    y1 = torch.log(a)
    y2 = torch.sin(x2)
    return y1*y2
```

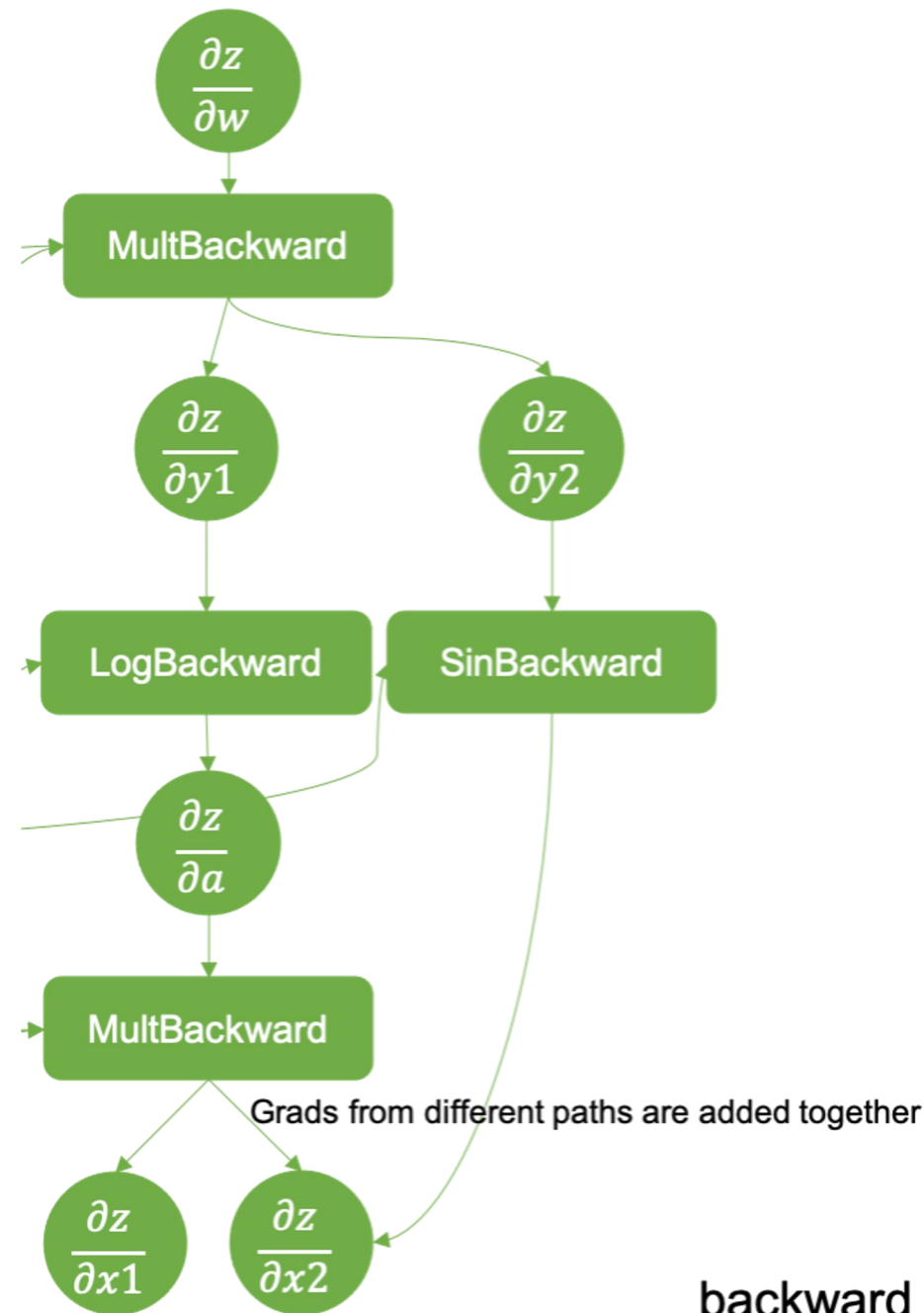


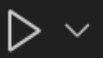
```
def get_graph(node, id=0, parent=None):
    if (parent is None):
        node = node.grad_fn
        tree_node = Node(f"{node.name()}:{id}")
    else:
        tree_node = Node(f"{node.name()}:{id}", parent=parent)
    for i, new_node in enumerate(node.next_functions):
        if (new_node[0] is not None):
            get_graph(new_node[0], id=i, parent=tree_node)
    return (tree_node)
```

```
w=fun(x1,x2) |
tree = get_graph(w)
for pre, _, node in RenderTree(tree):
    print(f"{pre}{node.name}")
```

[178] ✓ 0.0s

```
... MulBackward0:0
  └─ LogBackward0:0
    └─ MulBackward0:0
      └─ torch::autograd::AccumulateGrad:0
      └─ torch::autograd::AccumulateGrad:1
  └─ SinBackward0:1
    └─ torch::autograd::AccumulateGrad:0
```





```
x1 = torch.tensor(0.50,requires_grad=True)
x2 = torch.tensor(0.75,requires_grad=True)

def fun(x1,x2):
    a = x2*x1
    y1 = torch.log(a)
    y2 = torch.sin(x2)
    return y1*y2

w=fun(x1,x2)
w.backward()
print("Autodiff:", x1.grad, x2.grad)
with torch.no_grad():
    print("Manual:", torch.sin(x2)/x1, torch.sin(x2)/x2 + torch.log(x1*x2)*torch.cos(x2))
```

[317]

✓ 0.0s

Python

```
... Autodiff: tensor(1.3633) tensor(0.1912)
Manual: tensor(1.3633) tensor(0.1912)
```